

Orchestrating AI Agent

with Pattern Matching

Haim Michael

Founder & CEO @ Zindell Technologies
www.zindell.com | blog.lifemichael.com

life michael



Haim Michael

Made in Israel

My name is Haim Michael. I live in Tel Aviv, and I am the CEO & founder of Zindell Technologies, Ltd.

I have been into programming since my childhood. I have extensive experience with Java (30+ yrs), JavaScript (30+ yrs), C++ (30+ yrs), C# (20+ yrs), TypeScript (12+ yrs), Python (15+ yrs), PHP (20+ yrs), Scala (16+ yrs), and Kotlin (12+ yrs).



Passion for Learning

My passion lies in teaching and lifelong learning. I continuously learn and evolve.

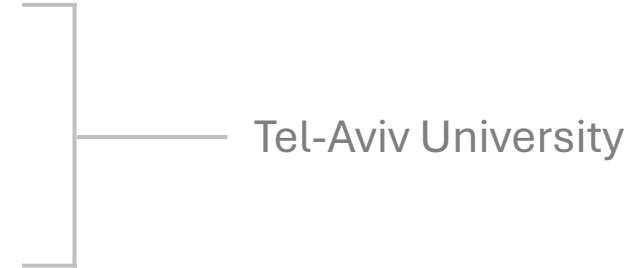
Learning takes place in various forms. Learning is not limited to the formal educational systems.

You thrive when you explore topics that genuinely interest you.

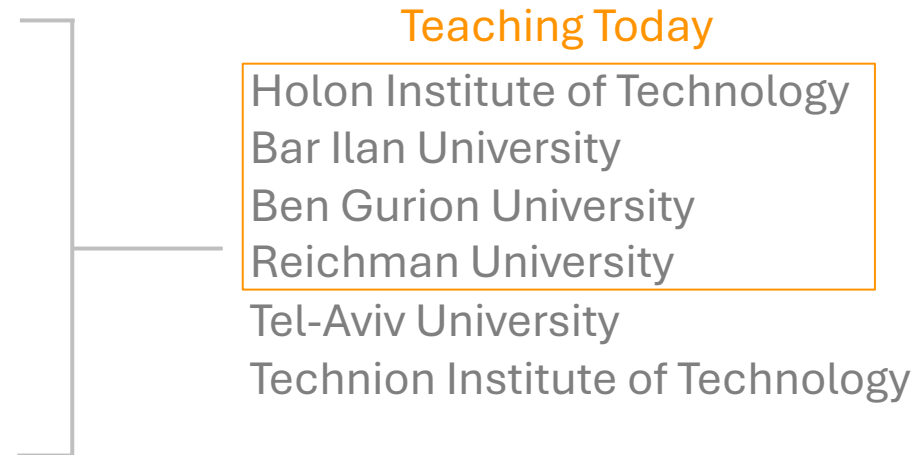
My Academic Background

B.Sc. in Economics & Accounting

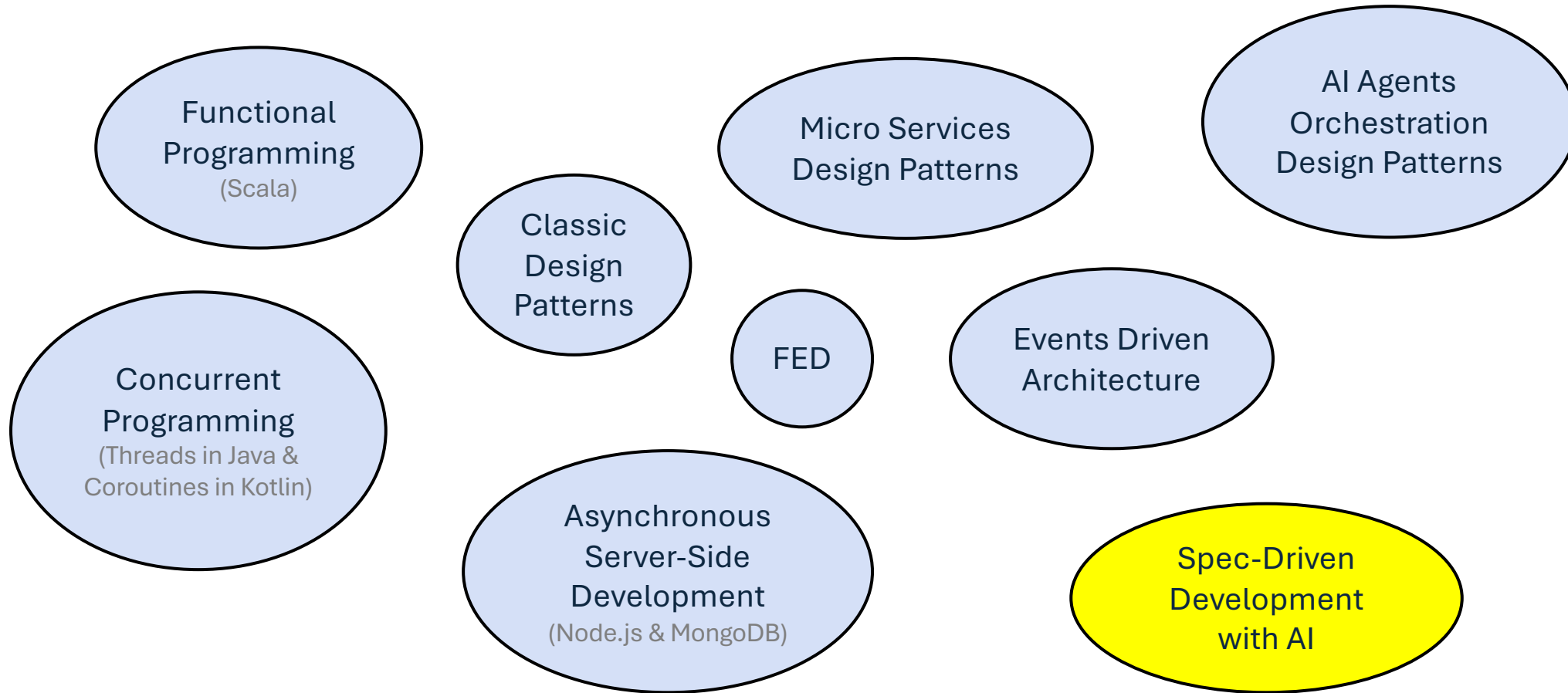
MBA (Information Systems Management)



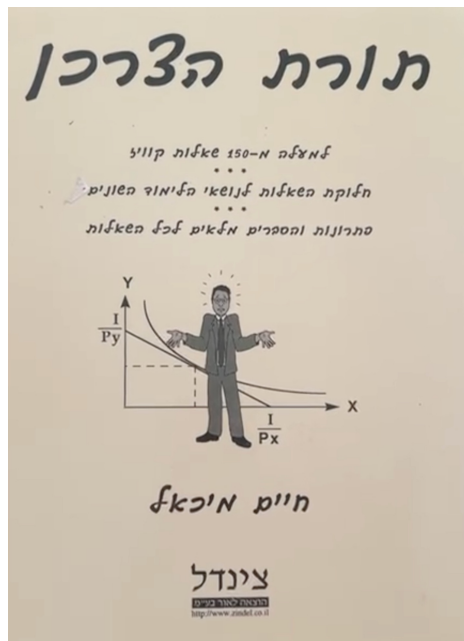
Teaching advanced topics in
software development at
leading academic institutions.



Academia

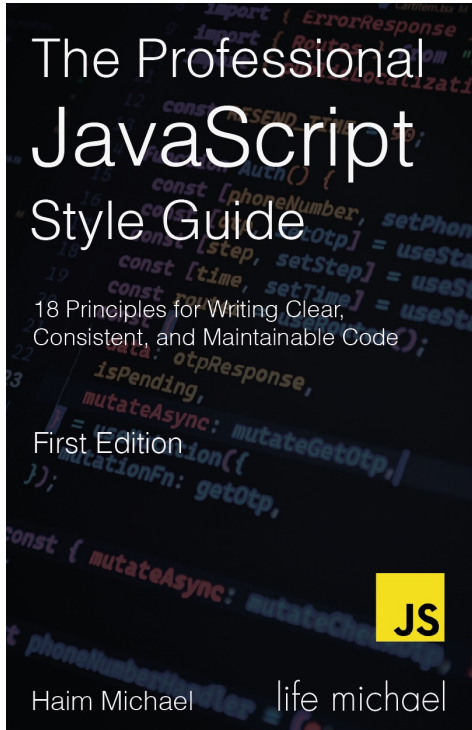


My Books

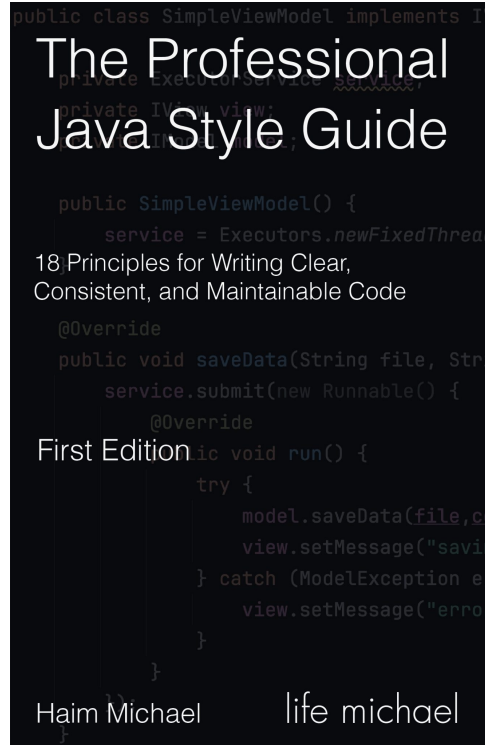
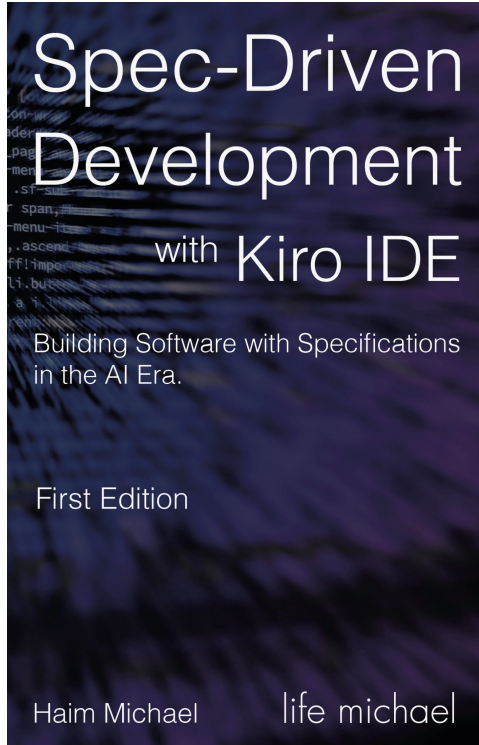


Teaching Micro Economic courses in the Open University

My Books



My Books



My Courses



למד בקצב שלך!

JS

תכנות בג'אווה סקריפט

למד לתכנת בג'אווה סקריפט, בקצב שלך ובזמן שלך. תמיכה לכל אורך הדרך באמצעות קבוצה סגורה בלינקדין.

הרשמה לקורס



למד בקצב שלך!

React

תכנות בג'אווה

למד לתכנת בג'אווה, בקצב שלך ובזמן שלך. תמיכה לכל אורך הדרך באמצעות קבוצה סגורה בלינקדין.

הרשמה לקורס



למד בקצב שלך!

Python

תכנות בפייתון

למד לתכנת בפייתון, בקצב שלך ובזמן שלך. תמיכה לכל אורך הדרך באמצעות קבוצה סגורה בלינקדין.

הרשמה לקורס



למד בקצב שלך!

TS

תכנות בטיפסקריפט

למד לתכנת בטיפסקריפט, בקצב שלך ובזמן שלך. תמיכה לכל אורך הדרך באמצעות קבוצה סגורה בלינקדין.

הרשמה לקורס

Courses (in Hebrew) @ <https://academy.lifemichael.com/he>

My Courses



Lambda Expressions in Kotlin

Acquire Practical Experience and In-depth Understanding!

life michael

Course

★ 4.6

12 ratings

1.5 total hours

10 lectures

€14.99



Inner Classes in Java

Learn about inner classes and nested classes in the Java...

life michael

Course

★ 4.4

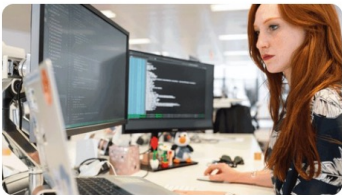
6 ratings

2 total hours

12 lectures

Expert

€14.99



Structural Pattern Matching in Python

Learn how to develop structural pattern matching in Python, an...

life michael

Course

★ 4.8

4 ratings

1 total hour

17 lectures

Intermediate

€14.99



Decorators in Python

Learn how to develop Decorators in Python!

life michael

Course

★ 3.9

23 ratings

1.5 total hours

10 lectures

Expert

€14.99

More Than 25 Online Courses on Udemy

Passion to Build

Jacado - More Than 200 Games for Mobile Telephones – Years 2001- 2008



Zindell - Building AI Aligned Companies – Years 2026 - ...

nana.events

nana.events



prompo

prompo.ai



ZINDREX

zindrex.com



ngager

ngager.pro



Jacado

jacado.games

Conferences & Meetups

XtremeJ

xtremej.dev

XtremePython

xtremepython.dev

XtremeJS

xtremejs.dev

XtremeAI

xtremeai.dev



meetup.com/lifemichael

More Than 4600 Members

Online Community Conferences, since 2021

Communities Builder



Python Developers

Professional Group on Facebook
More than 131000 Members

[facebook.com/groups/lifemichaelpython](https://www.facebook.com/groups/lifemichaelpython)



JavaScript Developers

Professional Group on Facebook
More than 35000 Members

<https://www.facebook.com/groups/407961892610345>



More Than 2800
Professional Video Clips
[youtube.com/lifemichael](https://www.youtube.com/lifemichael)

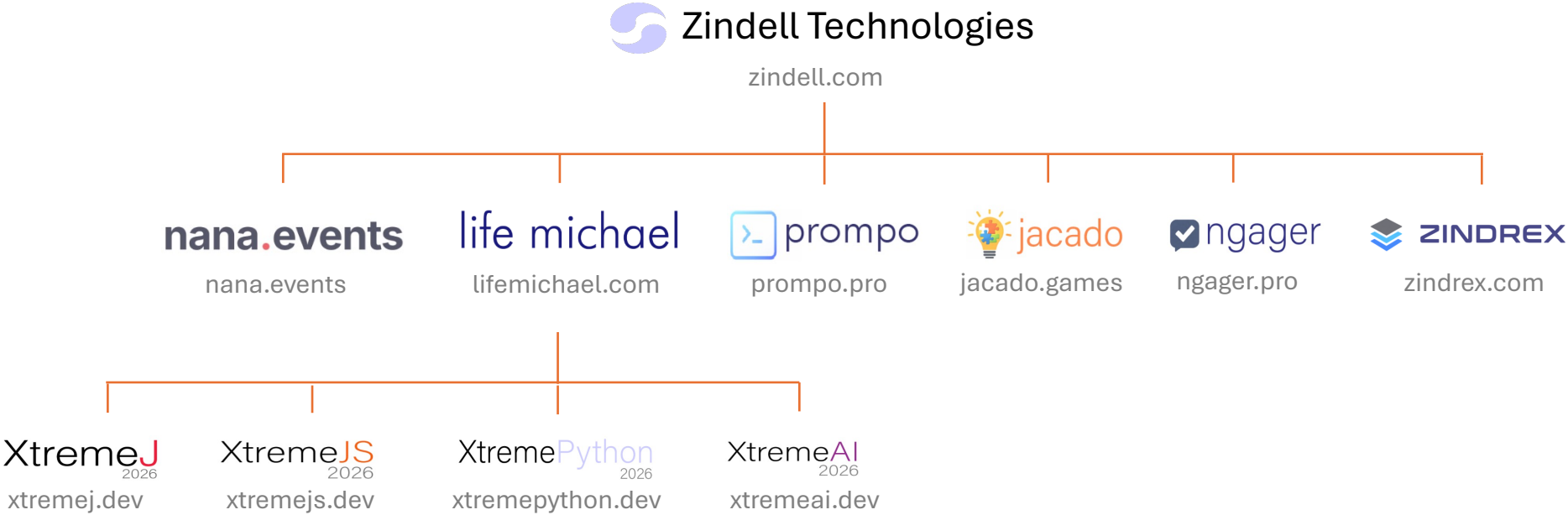


More Than 18000 Followers
<https://linkedin.com/in/lifemichael>

Monthly Newsletters



Zindell Technologies



Pattern Matching

What is Pattern Matching?

Pattern matching simplifies conditional logic by allowing developers to test and extract data from objects more concisely. Using Pattern Matching, we can eliminate boilerplate.

C#, Java, Kotlin, Scala, Swift & Python

The switch Expression

```
public class SimpleTypeMatching {  
    public static void main(String[] args) {  
        var animal = new Dog();  
        System.out.println(hello(animal));  
    }  
    public static String hello(Object animal) {  
        return switch(animal){  
            case Dog dog -> dog.hau();  
            case Tiger tiger -> tiger.miau();  
            case Cat cat -> cat.miau();  
            case null, default -> "hello";  
        };  
    }  
}
```

SimpleTypeMatching.java

The Completeness Requirement

When using pattern matching together with Switch Statement (or Switch Expressions), all possible values must be handled. Usually, we will achieve that using the `default` case and using a `sealed` class.

The Completeness Requirement

```
public class CompletenessRequirement {  
    public static void main(String[] args) {  
        var animal = new Dog();  
        System.out.println(hello(animal));  
    }  
    public static String hello(Animal animal) {  
        return switch(animal) {  
            case Dog dog -> dog.hau();  
            case Tiger tiger -> tiger.miau();  
            case Cat cat -> cat.miau();  
        };  
    }  
}
```

CompletenessRequirement.java

The Completeness Requirement

```
sealed abstract class Animal permits Dog, Cat {}
```

```
final class Dog extends Animal {  
    public String hau() {  
        return "hau";  
    }  
}
```

```
sealed class Cat extends Animal permits Tiger {  
    public String miau() {  
        return "miau";  
    }  
}
```

The Completeness Requirement

```
final class Tiger extends Cat {  
    public String miau() {  
        return "miaurrr";  
    }  
}
```

Guarded Patterns

Guarded patterns combine type matching with an additional condition, allowing a case to match only when both are satisfied.

Guarded Patterns

```
public class GuardedPatterns {  
  
    public static void main(String[] args) {  
        Object value = 15;  
  
        String result = switch (value) {  
            case Integer n when n > 10 -> "Large integer";  
            case Integer n -> "Small integer";  
            default -> "Not an integer";  
        };  
  
        System.out.println(result);  
    }  
}
```

GuardedPatterns.java

Record Deconstruction Pattern

We can easily use Pattern Matching with record classes to extract values from the object we compare.

Record Deconstruction Pattern

```
record Point(int x, int y) {}

public class DeconstructionDemo {
    public static void main(String[] args) {
        Object obj = new Point(10, 20);

        String result = switch (obj) {
            case Point(int x, int y) -> "x=" + x + ", y=" + y;
            default -> "Unknown";
        };

        System.out.println(result);
    }
}
```



DeconstructionDemo

Unnamed Variables & Patterns

We can easily use Pattern Matching with record classes to extract values from the object we compare.

Unnamed Variables & Patterns

```
public class UnnamedVariablesDemo {  
    public static void main(String[] args) {  
        int data = 12;  
        System.out.println(testType(data));  
    }  
    public static String testType(Object data) {  
        return switch (data) {  
            case String _ -> "java.lang.String";  
            case Integer _ -> "java.lang.Integer";  
            case Double _ -> "java.lang.Double";  
            default -> "Unknown";  
        };  
    }  
}
```

UnnamedVariablesDemo.java

Ai Agents Orchestration

Orchestrating AI Agent Results

```
public static void main(String[] args) {  
    ...  
    Agent agent = AiServices.create(Agent.class, model);  
    AgentRawResult raw = agent.chat("What is the capital of France?");  
    AgentResult result = raw.toAgentResult();  
    System.out.println(handle(result));  
}  
  
static String handle(AgentResult result) {  
    return switch (result) {  
        case AgentResult.Answer a -> "Answer: " + a.text();  
        case AgentResult.Clarification c -> "Clarification needed: " + c.question();  
        case AgentResult.Error e -> "Error: " + e.reason();  
    };  
}
```

Orchestrator.java

Pipeline (Planner->Coder->Reviewer) Orchestrator

```
while (!(current instanceof FinalResult)) {
    String agentName = switch (current) {
        case PlanRequest _    -> "Planner";
        case CodeRequest _    -> "Coder";
        case ReviewRequest _  -> "Reviewer";
        case FinalResult _    -> "Done";
    };

    try {
        current = step(current, planner, coder, reviewer);
    } catch (Exception e) {
        System.out.println(agentName + " " + agentName + "Agent failed - " + e.getMessage());
        return;
    }
}

FinalResult result = (FinalResult) current;
System.out.println("Orchestrator Final result: " + result.result());
```

———— PipelineOrchestrator.java

Branching Delegation (Router) Orchestrator

```
static String dispatch(TaskClassification task,
                      TranslatorAgent translator,
                      SummarizerAgent summarizer) {
    return switch (task) {
        case TranslateTask(var text, var lang) ->
            translator.translate("Translate the following into "
                                + lang + ": " + text);
        case SummarizeTask(var text) ->
            summarizer.summarize(text);
    };
}
```



RouterOrchestrator.java

Retry & Feedback Loop Orchestrator

```
switch (outcome) {
    case Approved a when a.score() > 7 -> {
        System.out.println("Reviewer Approved (high score: " + a.score() + ")");
        return a.code();
    }
    case Approved a -> {
        System.out.println("Reviewer Approved (score: " + a.score() + ")");
        return a.code();
    }
    case Rejected(var feedback) -> {
        if (attempt < MAX_ATTEMPTS) {
            System.out.println("Orchestrator, Re-invoking coder with feedback");
            code = coder.code(task + "\nFeedback: " + feedback);
        }
    }
}
```

RetryOrchestrator.java

Parallel Fan-Out Orchestrator

```
static String fanOut(String task, AgentA a, AgentB b, JudgeAgent judge) {  
    ...  
    String solA = futA.get(), solB = futB.get();  
    JudgeVerdict verdict = judge.judge(solA, solB);  
    return selectWinner(verdict, solA, solB);  
}  
  
static String selectWinner(JudgeVerdict verdict, String solutionA, String solutionB) {  
    return switch (verdict) {  
        case PickedA(var reason) -> solutionA + "\nChosen because: " + reason;  
        case PickedB(var reason) -> solutionB + "\nChosen because: " + reason;  
    };  
}
```

———— FanOutOrchestrator.java

Q & A

Thanks for attending my talk :)

WhatsApp +972.54.6655837

haim.michael@zindell.com

<https://blog.lifemichael.com>

You can download the code samples at

<https://tinyurl.com/aiagentspatterns>



XtremeJ
xtremej.dev

XtremeJS
xtremejs.dev

XtremePython
xtremepython.dev

XtremeAI
xtremeai.dev

nana.events

nana.events



jacado.games



zindrex.com



prompo.ai



ngager.pro

life michael

lifemichael.com



meetup.com/lifemichael